

Programación

Práctica de Alquileres de Coches

```

C:\> Símbolo del sistema - .\PRO_1 X + v
-----
                        ALQUILERES DE COCHES
1. Añadir cliente
2. Dar de baja cliente
3. Listar clientes
4. Añadir vehículo
5. Dar de baja vehiculo
6. Listar vehiculos
7. Abrir un alquiler
8. Cerrar un alquiler
9. Listar alquileres
0. Salir
Seleccionar una opción (0-9):
```

Contenido

Enunciado.....3

Fase 1.....8

 Modificaciones.....11

Fase 2.....12

Enunciado

Una empresa de alquiler de coches quiere digitalizar la gestión de la misma por lo que te piden que realices una aplicación para tal fin.

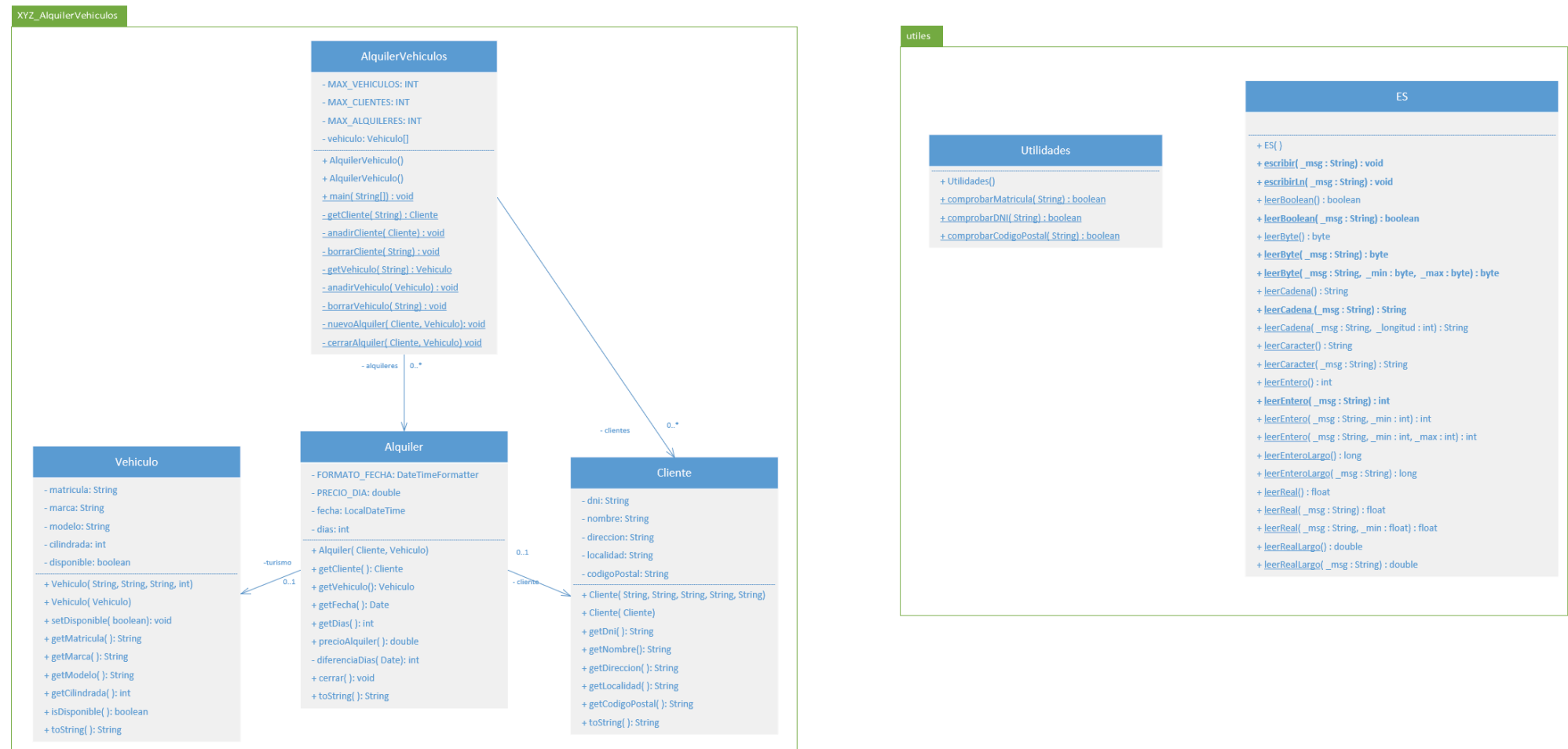
Por un lado, el cliente necesita controlar el alquiler de los vehículos, de los que guarda su matrícula, marca, modelo y cilindrada. Para los clientes guarda información sobre su nombre, dirección, localidad y código postal.

Cuando un cliente alquila un turismo se abre un nuevo alquiler en el que se almacena el cliente que lo alquila, el turismo que alquila, la fecha en que lo alquila y un contador de días que se pone a cero y que se actualizará cuando el cliente vaya a devolver el turismo y se cierre el alquiler.

El precio que cobra por alquiler para un vehículos es: $\text{PRECIO_DIA} \times \text{nº días} + \text{cilindrada} / 100$.

Tu tarea consiste en realizar una aplicación para gestionar el negocio de nuestro nuevo cliente siguiendo el diagrama de clases siguiente:

Para ello te pongo un diagrama de clases para el mismo y poco a poco te irá explicando los diferentes pasos a realizar:



utilis

Utilidades

+ Utilidades()

+ comprobarMatricula(String) : boolean

+ comprobarDNI(String) : boolean

+ comprobarCodigoPostal(String) : boolean

ES

+ ES()

+ escribir(_msg : String) : void

+ escribirLn(_msg : String) : void

+ leerBoolean() : boolean

+ leerBoolean(_msg : String) : boolean

+ leerByte() : byte

+ leerByte(_msg : String) : byte

+ leerByte(_msg : String, _min : byte, _max : byte) : byte

+ leerCadena() : String

+ leerCadena(_msg : String) : String

+ leerCadena(_msg : String, _longitud : int) : String

+ leerCaracter() : String

+ leerCaracter(_msg : String) : String

+ leerEntero() : int

+ leerEntero(_msg : String) : int

+ leerEntero(_msg : String, _min : int) : int

+ leerEntero(_msg : String, _min : int, _max : int) : int

+ leerEnteroLargo() : long

+ leerEnteroLargo(_msg : String) : long

+ leerReal() : float

+ leerReal(_msg : String) : float

+ leerReal(_msg : String, _min : float) : float

+ leerRealLargo() : double

+ leerRealLargo(_msg : String) : double

Práctica Alquiler de coches

Las tareas a realizar son:

- Crea en la clase Utilidades los métodos que permitan comprobar mediante una expresión regular si una matrícula, o código postal o un dni es válido.
- Importar y utilizar la clase ES desarrollada en tareas anteriores. Se han de añadir, al menos, los siguientes métodos a los ya creados:
 - o Crea los métodos estáticos leerEntero y leerReal que reciban como parámetro el mensaje que se ha de mostrar por pantalla y el valor mínimo que se puede leer desde el teclado. Si el usuario introduce un valor inferior al indicado ha de mostrar un error y volver a solicitar el dato. Una vez que el valor leído sea correcto lo devolverá.
- Crea la clase **Cliente**:
 - Los atributos han de ser privados.
 - Crea un constructor con cinco parámetros correspondientes a los cinco atributos de la clase. Crea también el constructor copia.
 - Crea los métodos get para cada uno de los atributos
 - Crea el método **toString** que devuelva los datos del cliente.
- Crea la clase **Vehiculo**:
 - Los atributos han de ser privados.
 - Crea un **constructor** con cuatro parámetros correspondientes a los cuatro atributos de la clase y que los inicialice adecuadamente. Por defecto, un nuevo turismo estará disponible cuando se cree el objeto. Crea también el **constructor copia**.
 - Crea los métodos **get** para los atributos.
 - Crea el método **toString** que devuelva los datos del turismo.
- Crea la clase **Alquiler**:
 - Añade los atributos con la visibilidad adecuada.
 - Crea un **constructor** con dos parámetros que además de inicializar los atributos cliente y vehículo, también inicialice la fecha a la fecha actual, días a cero y ponga el turismo como no disponible.

Práctica Alquiler de coches

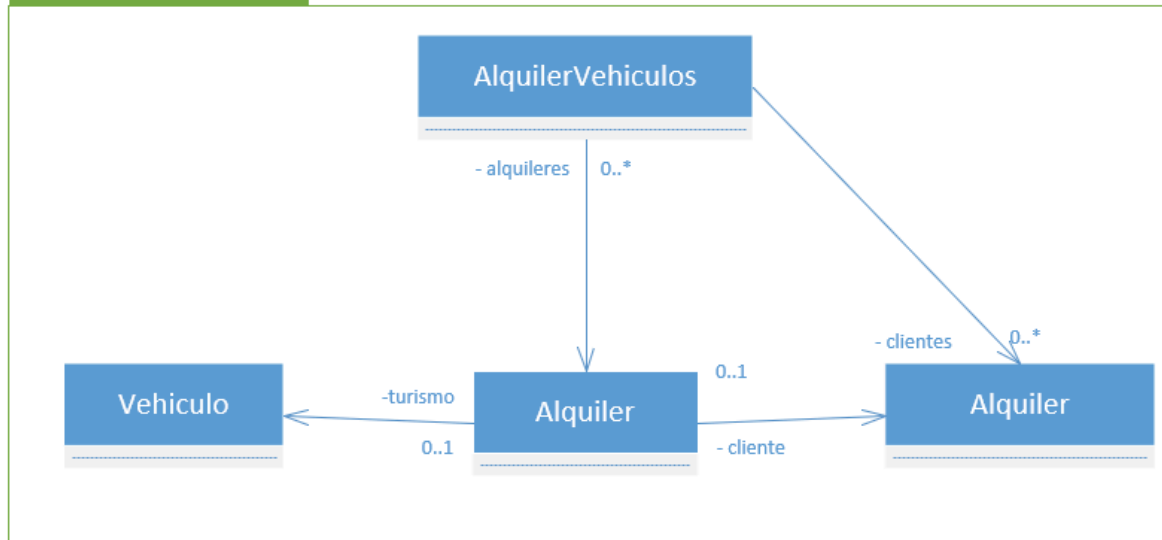
- Crea los métodos **get** para los atributos.
 - Crea un método **cerrar** que cerrará el alquiler, para lo que, partiendo de la fecha actual y la fecha en que se realizó el alquiler, calcule el número de días (si se devuelve el mismo día contará como 1 día), y ponga el turismo como disponible.
 - Crea un método privado llamado **diferenciaDias** y que devuelva la diferencia en días entre dos fechas y luego utilízalo en el método cerrar.
 - Crea un método **precioAlquiler** que devolverá el precio del alquiler basándote en la regla de negocio que nos ha comentado el cliente.
 - Crea el método **toString** que devuelva los datos del alquiler.
-
- Crea la clase **AlquilerVehiculos** con sus atributos y visibilidad adecuados.
 - Crea un método **getClient** que se le pase un DNI y nos lo devuelva si este existe un cliente con dicho DNI o null en caso contrario.
 - Crea un método **anadirCliente** que añada un cliente al array de clientes si cabe y si no existe ningún otro con el mismo DNI o muestre un mensaje con el error que se ha producido.
 - Crea un método **borrarCliente** que elimine un cliente, dado su DNI.
 - Crea un método **getVehiculo** que se le pase la matrícula de un turismo y nos lo devuelva si este existe o null en caso contrario.
 - Crea un método **anadirVehiculo** que añada un coche al array de vehiculos si cabe y no existe ningún otro con la misma matrícula o muestre un mensaje con el error que se ha producido.
 - Crea un método **borrarVehiculo** que borre un vehiculo, dada su matrícula, del array de vehiculos.
 - Crea un método **nuevoAlquiler** que dado un cliente y un vehiculo cree un nuevo alquiler y lo añada al array de alquileres. Para ello se debe comprobar que el vehiculo esté disponible.
 - Crea un método **cerrarAlquiler** que cierre el alquiler dado un cliente y un vehiculo.
 - Crea un método **main**. El método main deberá mostrarnos un menú con las diferentes opciones (añadir cliente, borrar cliente, listar clientes, añadir vehículo, borrar vehículo, listar vehículo, abrir un alquiler, cerrar un alquiler, listar alquileres y salir) y actuar en consecuencia. El menú se repetirá mientras no elijamos la opción salir. En todo caso se debe validar que todas las entradas al programa son correctas. Este método realizará llamadas a los distintos métodos de la clase **ES** para mostrar la información por pantalla o solicitar datos desde teclado. De igual manera realizará llamadas a los distintos métodos de la clase **Utilidades** para comprobar que los dnis, matrículas y códigos postales tengan un formato adecuado.
 - El máximo números de vehículos, clientes y alquileres que se permitirán sera de 50.

Práctica Alquiler de coches

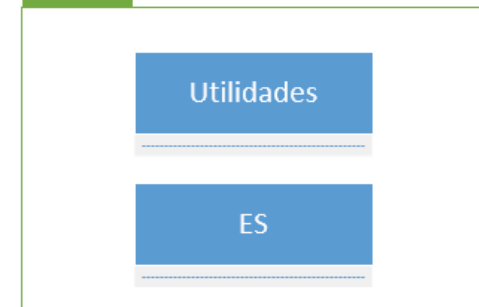
- Realiza la gestión de excepciones en los métodos que sean necesarios.

Fase 1

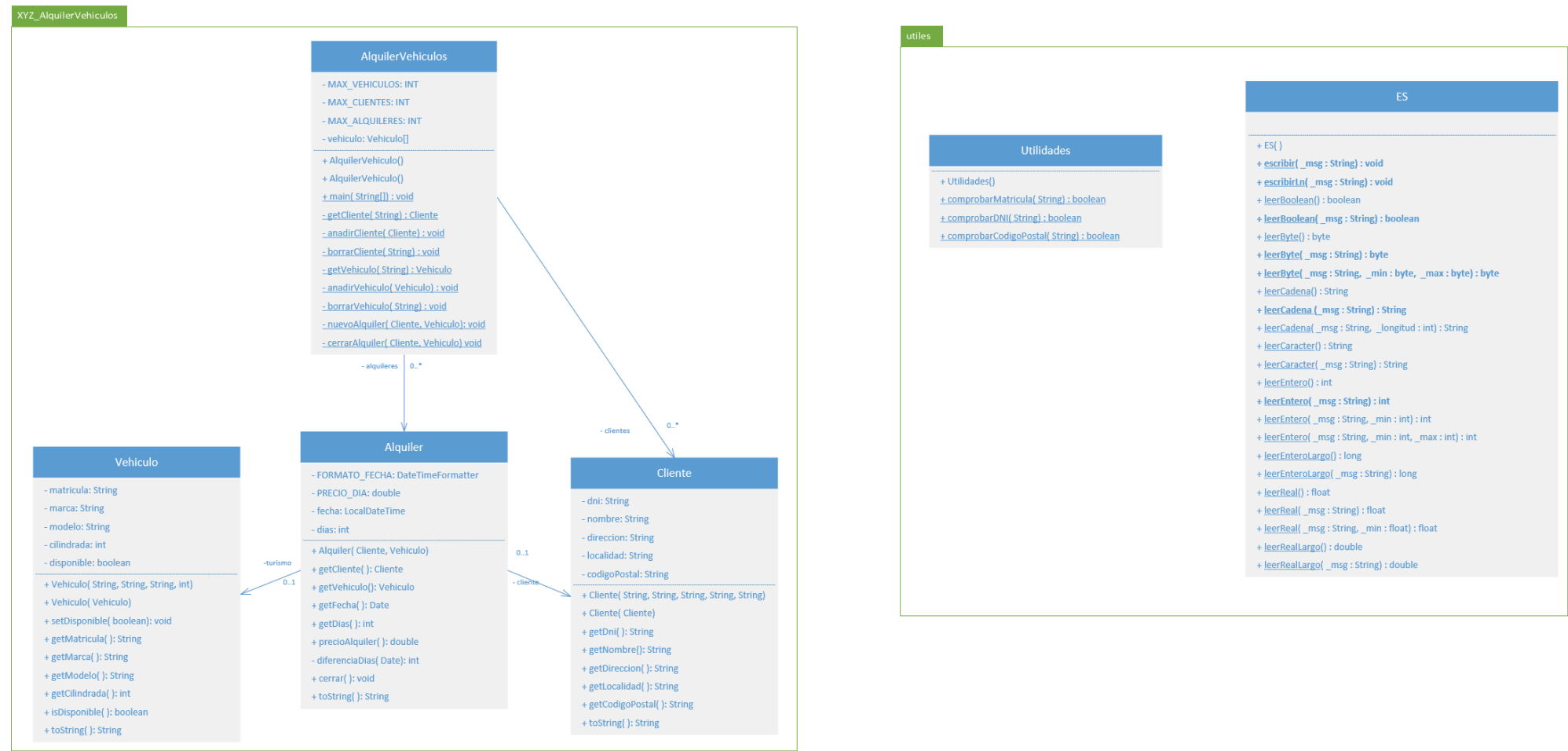
XYZ_AlquilerVehiculos



utiles



Práctica Alquiler de coches



utilis

Utilidades

+ Utilidades()

+ comprobarMatricula(String) : boolean

+ comprobarDNI(String) : boolean

+ comprobarCodigoPostal(String) : boolean

ES

+ ES()

+ escribir(_msg : String) : void

+ escribirLn(_msg : String) : void

+ leerBoolean() : boolean

+ leerBoolean(_msg : String) : boolean

+ leerByte() : byte

+ leerByte(_msg : String) : byte

+ leerByte(_msg : String, _min : byte, _max : byte) : byte

+ leerCadena() : String

+ leerCadena(_msg : String) : String

+ leerCadena(_msg : String, _longitud : int) : String

+ leerCaracter() : String

+ leerCaracter(_msg : String) : String

+ leerEntero() : int

+ leerEntero(_msg : String) : int

+ leerEntero(_msg : String, _min : int) : int

+ leerEntero(_msg : String, _min : int, _max : int) : int

+ leerEnteroLargo() : long

+ leerEnteroLargo(_msg : String) : long

+ leerReal() : float

+ leerReal(_msg : String) : float

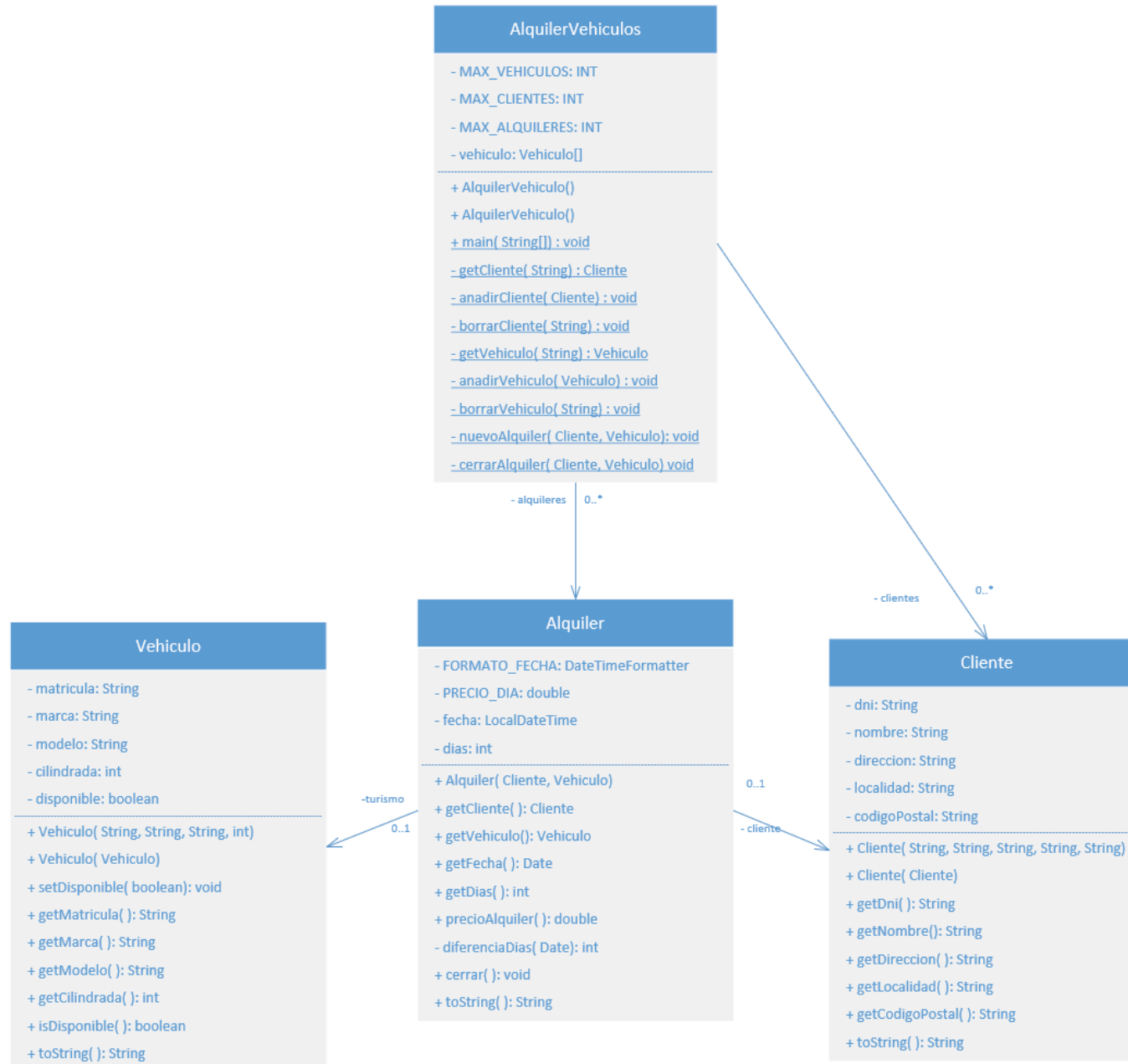
+ leerReal(_msg : String, _min : float) : float

+ leerRealLargo() : double

+ leerRealLargo(_msg : String) : double

Práctica Alquiler de coches

XYZ_AlquilerVehiculos



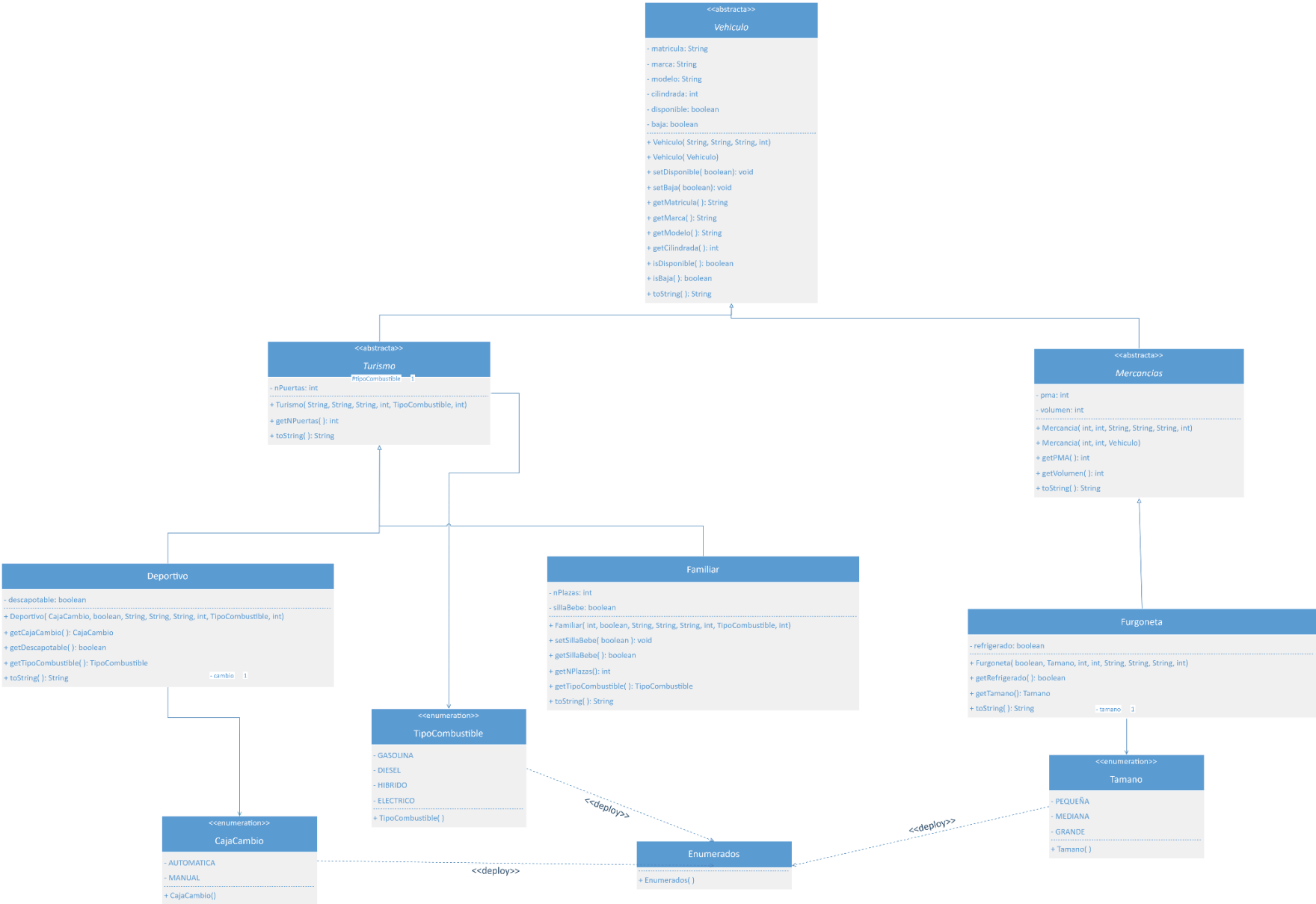
Modificaciones

Incorporar el atributo `baja` a las clases `Vehiculo` y `Cliente`. Realizar los siguientes métodos:

- `public boolean isBaja()`
- `public void setBaja( boolean baja)`

Fase 2

Debido al buen funcionamiento de la aplicación creada y de la empresa en general, ha decidido ampliar la flota por lo que necesita realizar una serie de modificaciones en la estructura de clases que tenía el programa. La modificación y ampliación de las mismas se puede ver en el siguiente diagrama de clases:



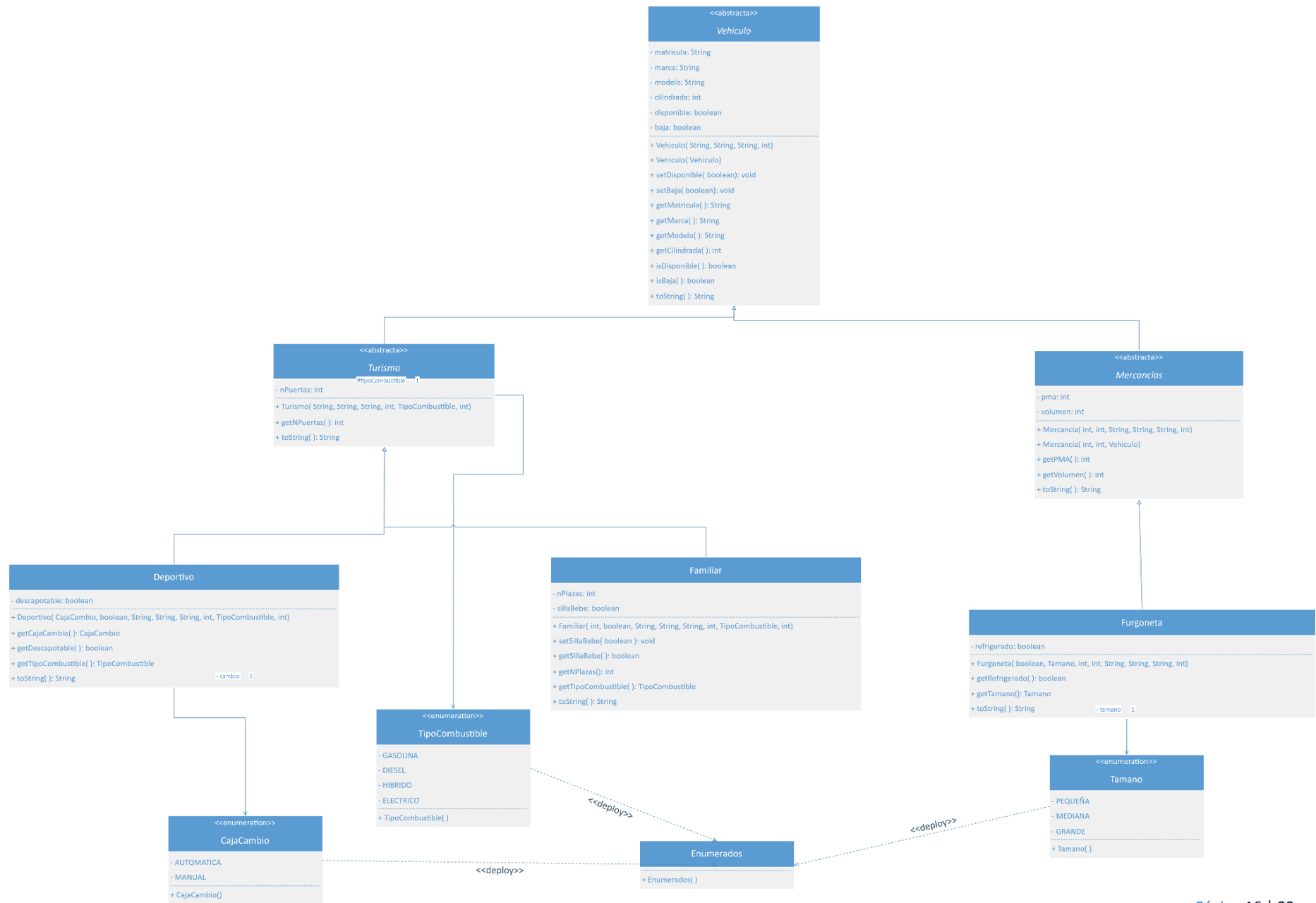
A partir del diagrama de clases, realizar las siguientes tareas:

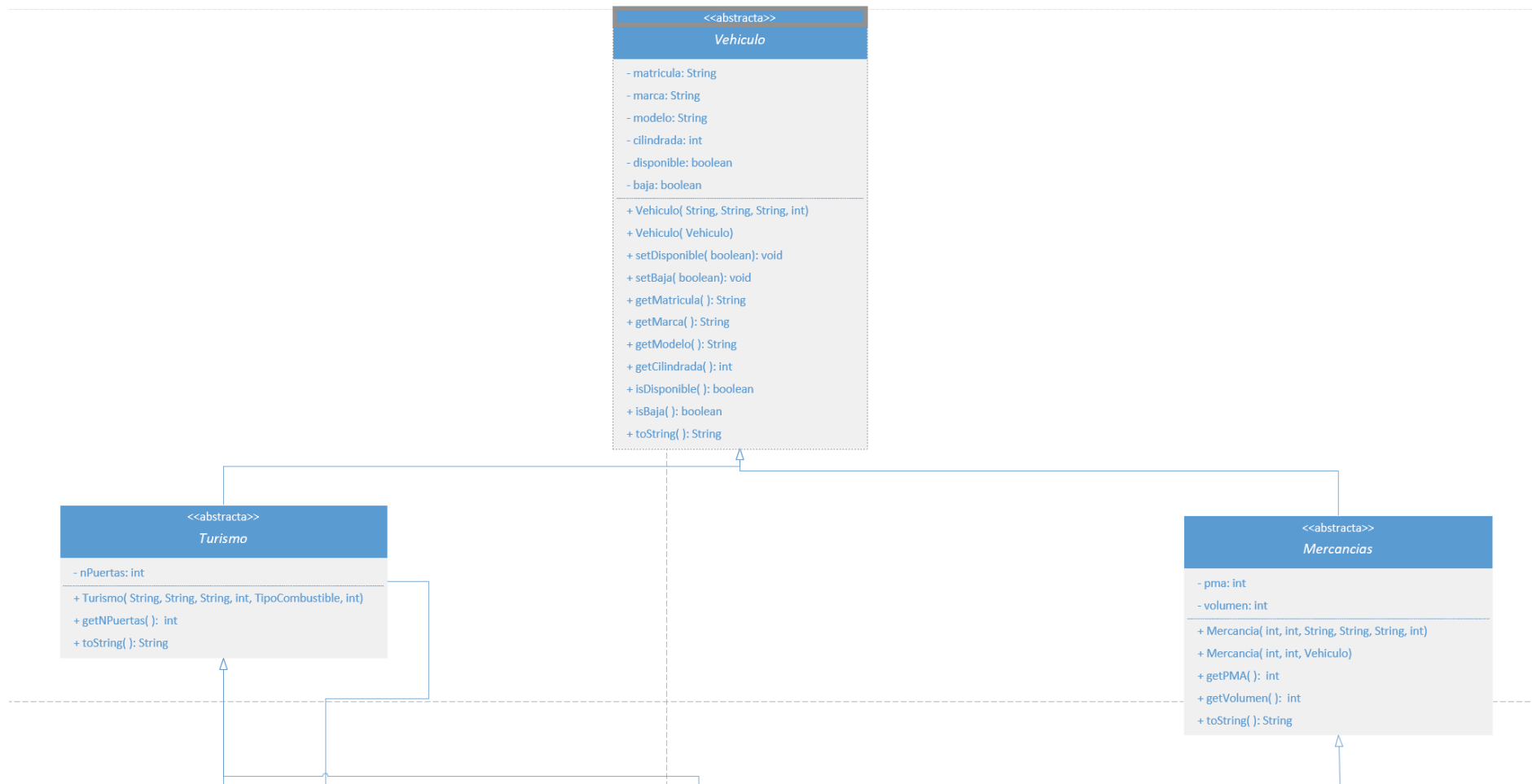
- Modificar la clase **Vehiculo** para que esta sea abstracta, manteniendo los atributos y métodos creados en la anterior práctica.
- Crear la clase **Turismo** que ha de heredar de Vehiculo también ha de ser abstracta.
 - Crear los atributos *nPuertas* y *combustible*. Combustible es de tipo enumerado (se explica a continuación). Los atributos han de ser protected.
 - Crear un constructor que reciba todos los parámetros para crear el objeto. Ha de llamar al constructor padre mediante el uso de *super*.
 - Crear el método toString que muestre todos los datos (tanto los de la clase Turismo como la de Vehiculo).
- Crear la clase **Deportivo** que hereda de Turismo. De la clase Deportivo no se puede heredar.
 - Crear los atributos *descapotable* y *cambio* (tipo enumerado CajaCambios), que sean privados.
 - Crear un constructor que reciba todos los parámetros para crear el objeto. Ha de llamar al constructor padre mediante el uso de *super*.
 - Crear los métodos get para los atributos de la clase Turismo y Deportivo.
 - Crear el método toString que muestre todos los datos almacenados.
- Crea la clase **Familiar**, de forma que no pueden heredar de ella, y cuyo padre sera la clase **Turismo**.
 - Crear los atributos *nPlazas* (ha de ser un número positivo comprendido entre 4 y 7) y *sillaBebe*, que han de ser privados.
 - Crear un constructor que reciba todos los parámetros para crear el objeto. Ha de llamar al constructor padre mediante el uso de *super*.

Práctica Alquiler de coches

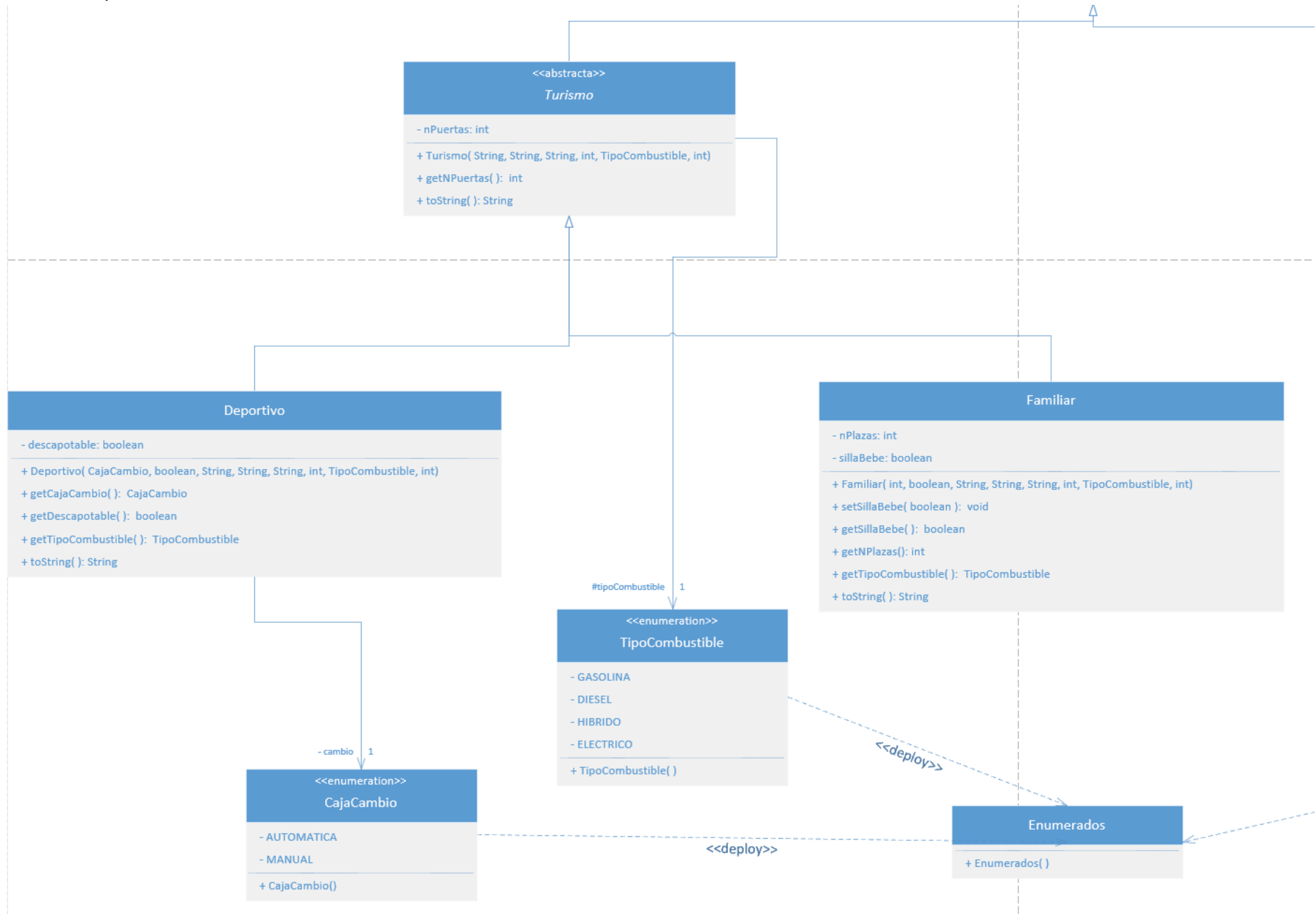
- Crear los métodos *get* para los atributos de la clase Turismo y Familiar.
- Crear el método *toString* que muestre todos los datos almacenados
- Crear la clase **Mercancias** que es abstracta y hereda de Vehiculo.
 - Crear los atributos *pma* (Peso Máximo Autorizado) y *volumen*. Los valores introducidos han de ser positivos.
 - Crear los constructores descritos en el diagrama de clases.
 - Implementar los métodos *get* para los atributos de la clase y el método *toString* que muestre todos los valores.
- Crear la clase **Furgoneta** que hereda de Mercancias.
 - Crear los atributos *refrigerado* y *tamano* de tipo enumerado.
 - Crear los constructores descritos en el diagrama de clases.
 - Implementar los métodos *get* para los atributos de la clase y el método *toString* que muestre todos los valores.
- Crear la clase **Enumerados** que contendrá los distintos tipos enumerados que se utilizan en el ejercicio.
 - TipoCombustible. Tiene los siguientes datos: gasolina, diesel, hibrido y electrico.
 - CajaCambios. Tiene los siguientes datos: automatica y manual.
 - Tamano. Tiene los siguientes datos: grande, mediana y pequena.
- Modificar la clase principal de la tarea de la unidad anterior que permita trabajar con los distintos tipos de vehículos que hemos creado.

Práctica Alquiler de coches





Práctica Alquiler de coches



<<deploy>>

